

Assignment - 11

CODES:

[conftest.py](#)

```
import pytest
from selenium import webdriver

@pytest.fixture(scope="class")
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.implicitly_wait(10)
    yield driver
    driver.quit()
```

[pages.py](#)

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Exercise 4: Component-Based Architecture
class SideMenuComponent:
    def __init__(self, driver):
        self.driver = driver
        self.pim_link = (By.XPATH, "//span[text()='PIM']")
        self.admin_link = (By.XPATH, "//span[text()='Admin']")

    def select_pim(self):
        self.driver.find_element(*self.pim_link).click()

# Exercise 1 & 2: Basic POM & Sync
# assignment_11/pages.py

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.url = "https://opensource-demo.orangehrmlive.com/"
        self.user_field = (By.NAME, "username")
        self.pass_field = (By.NAME, "password")
        self.login_btn = (By.TAG_NAME, "button")

    def load(self):
        self.driver.get(self.url)
```

```

def login(self, user, pwd):
    # ADD THIS: Wait explicitly for the username field to be present
    WebDriverWait(self.driver, 10).until(
        EC.presence_of_element_located(self.user_field)
    )

    self.driver.find_element(*self.user_field).send_keys(user)
    self.driver.find_element(*self.pass_field).send_keys(pwd)
    self.driver.find_element(*self.login_btn).click()
    return DashboardPage(self.driver)

class DashboardPage:
    def __init__(self, driver):
        self.driver = driver
        self.menu = SideMenuComponent(driver) # Component instance
        # Exercise 2: Wait for Dashboard Heading
        WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located((By.XPATH,
            "//h6[text()='Dashboard']"))
        )

# Exercise 3 & 5: Navigation, Chaining, and List Elements
class PIMPage:
    def __init__(self, driver):
        self.driver = driver
        # Exercise 5: List elements locator
        self.user_rows = (By.XPATH,
            "//div[@class='oxd-table-card']//div[@role='cell'] [3]/div")

    def verify_user_exists(self, target_name):
        try:
            # 1. Wait for at least one cell to be visible before checking
            WebDriverWait(self.driver, 7).until(
                EC.visibility_of_element_located(self.user_rows)
            )

            # 2. Get all elements matching the column
            elements = self.driver.find_elements(*self.user_rows)

            # 3. Check if the text matches
            return any(target_name.lower() in el.text.lower() for el in
            elements)
        except:
            # If the table doesn't load or elements aren't found, return False
            # instead of crashing
            return False

```

test_orange_hrm.py

```
import pytest
import pytest_check as check
from pages import LoginPage, PIMPage

class TestOrangeHRM:

    @pytest.fixture(autouse=True, scope="class")
    def setup_class(self, driver):
        # Attach variables to the Class itself
        TestOrangeHRM.driver = driver
        TestOrangeHRM.login_page = LoginPage(driver)

    def test_exercise_1_and_2_login(self):
        """Exercise 1 & 2: Login and Sync."""
        self.login_page.load()
        # Use TestOrangeHRM. instead of self.
        TestOrangeHRM.dashboard = self.login_page.login("Admin", "admin123")
        assert "dashboard" in self.driver.current_url.lower()

    def test_exercise_3_and_4_navigation(self):
        """Exercise 3 & 4: Sidebar and Chaining."""
        # Retrieve from Class level
        self.dashboard.menu.select_pim()
        TestOrangeHRM.pim_page = PIMPage(self.driver)
        assert "viewEmployeeList" in self.driver.current_url

    def test_exercise_5_data_verification(self):
        """Exercise 5: List Elements."""
        # Retrieve from Class level
        target_user = "Charlie"
        check.is_true(self.pim_page.verify_user_exists(target_user), f"User {target_user} missing!")

        print("Execution continued!")
```

OUTPUT:

```
(.venv) PS C:\Wipro Training\Assignments> pytest -v assignment_11/test_orange_hrm.py
=====
test session starts
=====
=
```

platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Wipro Training\Assignments\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Wipro Training\Assignments
plugins: check-2.8.0
collected 3 items

assignment_11/test_orange_hrm.py::TestOrangeHRM::test_exercise_1_and_2_login
PASSED [33%]
assignment_11/test_orange_hrm.py::TestOrangeHRM::test_exercise_3_and_4_navigation PASSED [66%]
assignment_11/test_orange_hrm.py::TestOrangeHRM::test_exercise_5_data_verification
FAILED [100%]

=====
===== FAILURES
=====

TestOrangeHRM.test_exercise_5_data_verification

FAILURE: check bool(False): User Charlie missing!
assignment_11/test_orange_hrm.py:32 in test_exercise_5_data_verification() ->
check.is_true(self.pim_page.verify_user_exists(target_user), f"User {target_user}
missing!")

Failed Checks: 1

----- Captured stdout call

Execution continued!

=====

short test summary info

=====

FAILED

assignment_11/test_orange_hrm.py::TestOrangeHRM::test_exercise_5_data_verification
- check bool(False): User Charlie missing!

===== 1
failed, 2 passed in 11.95s

=====

It's just an assertion error which is correct according to the code.
No actual error